

C PROGRAMMING

01 - Structure of a C Program

🎯 **Aim:** To understand the standard sections every C program is built from, in the order the compiler expects them, and why each section exists.

1. WHY DOES A C PROGRAM NEED A "STRUCTURE"?

C is not a free-form language — the compiler expects code to be organised into specific sections, in a specific order. Learning this fixed skeleton means you can look at **any** C program, even one you've never seen, and immediately recognise where the includes are, where variables live, and where execution actually starts.

2. THE SIX SECTIONS OF A C PROGRAM

1. Documentation Section

Comments describing what the program does, author name, date. Optional but good practice.

2. Preprocessor / Link Section

#include statements that bring in library code, e.g. #include <stdio.h>.

3. Global Declaration Section

Variables or function prototypes declared outside main(), visible to every function in the file.

4. main() Function Section

int main() { ... } — the compulsory entry point. Execution always begins here.

5. Local Declaration & Statements

Variables declared inside main() (or any function), followed by the actual instructions (executable statements).

6. User-Defined Function Section

Any additional functions you write yourself, usually placed after main().

3. ANNOTATED EXAMPLE

```
// Documentation Section
// Program: Add two numbers
// Author: Voflic Academy

/* Preprocessor / Link Section */
#include <stdio.h>

/* Global Declaration Section */
int total; // visible to every function below

/* main() Function Section */
int main() {
    // Local Declaration
    int a = 5, b = 10;

    // Executable Statements
    total = a + b;
    printf("Total = %d\n", total);

    return 0;
}
```

Documentation is written as comments — the compiler skips these entirely, they exist purely for humans reading the code.

#include <stdio.h> must appear before you use `printf()`/`scanf()`, since the compiler needs to know these functions exist before it sees them being called.

int total; declared outside `main()` is a **global variable** — it exists for the entire lifetime of the program and can be accessed inside any function, unlike a local variable which only exists inside the function it's declared in.

int main() is the only section that is truly **compulsory** — every single C program must have exactly one `main()`, because that's where the operating system hands control to your program.

return 0; tells the operating system the program finished successfully. A non-zero return generally signals that something went wrong.

💡 What is actually compulsory vs optional?

- **Compulsory:** the `main()` function. A C program with no `main()` will not compile into a runnable program.
- **Practically necessary:** `#include <stdio.h>` — only if you use input/output functions like `printf/scanf`.
- **Optional:** documentation comments, global variables, and user-defined functions — a tiny program can consist of nothing but an empty `main()`.

Curly braces { } define a "block."

Everything between the opening { and closing } of `main()` belongs to that function. Every opening brace must have a matching closing brace — a missing one is one of the most common beginner compile errors (covered in Topic 16).

📖 **One-Line Revision: Documentation → Preprocessor → Global Declarations → `main()` → Local Declarations & Statements → User-Defined Functions — in that order, with `main()` as the only compulsory piece.**

C PROGRAMMING

02 - Compilation and Execution

🎯 **Aim:** To understand the journey a C program takes from human-readable source code to a running program — the stages a compiler performs, and the commands used to trigger them.

1. WHY CAN'T THE COMPUTER RUN C CODE DIRECTLY?

A CPU only understands **machine code** — raw binary instructions specific to its hardware. The C code we write (like `printf("Hello");`) is designed for humans to read, not for the CPU. A special program called a **compiler** (e.g. GCC — the GNU Compiler Collection) translates our source code into machine code the CPU can actually execute.

2. THE FOUR STAGES OF COMPILATION

1 Preprocessing

Handles all lines starting with `#` before real compilation begins.

```
program.c → program.i
```

Expands `#include` files (copies in the full content of `stdio.h`, etc.) and replaces `#define` macros with their literal values.

2 Compilation

Translates the preprocessed C code into Assembly language.

```
program.i → program.s
```

This is also where the compiler checks for **syntax errors** — missing semicolons, undeclared variables, mismatched types, etc.

3 Assembly

Converts the Assembly code into machine-readable Object code.

```
program.s → program.o
```

Object code is binary, but not yet a complete, runnable program — pieces are still missing.

4 Linking

Combines your object code with pre-compiled library code (like the real implementation of `printf`).

```
program.o + libraries → program (executable)
```

This is where **linker errors** appear — e.g. calling a function that was declared but never defined anywhere.

Source Code (.c) → Preprocessor → Compiler → Assembler → Linker → Executable File

3. COMPILING & RUNNING WITH GCC (COMMAND LINE)

Step 1: Compile the source file into an executable
`gcc program.c -o program`

```
# Step 2: Run the executable
```

```
# On Linux / macOS:
```

```
./program
```

```
# On Windows:
```

```
program.exe
```

gcc → invokes the GNU C Compiler.

program.c → the source file to compile.

-o program → "output" flag; names the resulting executable file program (without -o, GCC defaults to naming it a.out).

./program → the ./ tells the shell "run the file named program located right here in this folder," which is required on Linux/macOS for safety reasons.

⚠ **Compile-time vs Run-time**

Compile-time is everything that happens during the gcc program.c -o program step — syntax errors are caught here, and the program never even gets created if these fail. **Run-time** is everything that happens after you launch ./program — a program can compile perfectly but still crash or misbehave at run-time (e.g. dividing by zero, reading beyond an array).

What if I don't have GCC installed?

You can also compile and run C instantly using free **online compilers** in a browser — no installation needed. This is exactly what the next topic, "Interactive and Script Mode," compares against writing and compiling a saved .c file yourself.

 **One-Line Revision: Preprocessing → Compilation → Assembly → Linking turns your .c file into a runnable program; gcc file.c -o output then ./output is the standard command-line workflow.**

C PROGRAMMING

03 - Interactive and Script Mode

🎯 **Aim:** To understand the two common ways of writing and running a C program — quick interactive/online compilation versus the traditional script (file-based) workflow — and when to use each.

1. TWO WAYS TO WORK WITH C

Unlike some languages (like Python) that have a true line-by-line interactive shell, C is always compiled as a whole before it runs. So "interactive mode" for C usually refers to using an **online/browser-based compiler** that hides the compile step from you, versus writing a real **.c script file** on disk and compiling it yourself in a terminal — the professional, real-world workflow.

🌐 Interactive / Online Mode

Type code directly into a website (e.g. an online C compiler/IDE), click "Run," and see output instantly — no installation, no terminal.

Good for:

- Quick experiments, testing a single concept
- Students without a compiler installed on their PC
- Sharing a short snippet with someone else instantly

Limitations:

- Not suitable for multi-file, real-world projects
- Depends on internet access
- Doesn't teach the real compile/run commands used on the job

📄 Script (File-Based) Mode

Write code in a text editor, save it as a **.c** file on your computer, then compile and run it using terminal commands (as shown in Topic 2).

Good for:

- Real projects with multiple files
- Learning the actual professional workflow (editor + compiler + terminal)
- Works completely offline once the compiler is installed

Limitations:

- Requires installing a compiler (like GCC) first
- A few extra steps (save, compile, run) compared to clicking one "Run" button

2. THE SCRIPT MODE WORKFLOW, STEP BY STEP

```
# 1. Write your code in an editor (VS Code, Notepad++, etc.)
# and save it with a .c extension
program.c

# 2. Open a terminal in the same folder and compile
gcc program.c -o program

# 3. Run the compiled executable
./program
```

This is exactly the three-step cycle you will repeat every single time you write and test a C program from now on: **edit → compile → run**.

If step 2 shows errors, you go back to step 1, fix them, and compile again — the executable from step 3 is only created if compilation succeeds cleanly.

This same cycle scales up naturally to large real-world software projects with hundreds of files — online compilers cannot realistically handle that scale.

💡 **Which one should Voflic students use?**

Use an **online compiler** when you just want to quickly test a small idea during class. But for serious practice and assignments, switch to the **script mode** workflow (a real code editor + GCC installed locally) as early as possible — it's what you'll use in every real job, internship, and larger project, and it forces you to get comfortable with the terminal and the actual compile commands from Topic 2.

📄 **One-Line Revision: Interactive/online mode = fast, no setup, good for quick tests; Script mode = save a real .c file and compile/run it yourself in a terminal — the standard professional workflow.**

C PROGRAMMING

04 - Comments and Documentation

🎯 **Aim:** To learn how to write comments in C, why documenting code matters, and the rules that keep comments from breaking your program.

1. WHAT IS A COMMENT?

A comment is text in your source code that the **compiler completely ignores**. It exists purely for humans — to explain what a piece of code does, why a decision was made, or to temporarily disable a line while testing. C supports two comment styles.

1. Single-Line Comment //

Everything from // to the end of that line is ignored.

```
int age = 20; // stores the student's age
```

2. Multi-Line Comment /* */

Everything between /* and */ is ignored, even across several lines.

```
/* This program calculates  
the average of three numbers  
entered by the user. */
```

2. EXAMPLE IN CONTEXT

```
/* ----- Program: Simple Interest Calculator Author:  
Voflic Academy ----- */  
#include <stdio.h>  
  
int main() {  
    float p, r, t, si; // p=principal, r=rate, t=time  
  
    printf("Enter principal, rate, time: ");  
    scanf("%f%f%f", &p, &r, &t);  
  
    // Simple Interest formula: SI = (P * R * T) / 100  
    si = (p * r * t) / 100;  
  
    // printf("Debug: si = %f\n", si); ← disabled line, kept for later debugging  
    printf("Simple Interest = %.2f\n", si);  
  
    return 0;  
}
```

The block comment at the top is **documentation** — a header describing the program's purpose and author, a common professional convention.

The single-line comment after variable declarations explains what each short variable name (p, r, t) actually stands for — very useful since single-letter names aren't self-explanatory.

The comment above the formula explains **why** the calculation looks the way it does — this kind of comment is more valuable than one that just repeats what the code obviously does.

The commented-out `printf("Debug: ...")` shows a very common real technique: temporarily "turning off" a line of code without deleting it, often used while debugging (see Topic 16).

⚠ Rules and Common Mistakes

- **Comments cannot be nested** — writing `/* outer /* inner */ still-outer */` will cause an error, because the first `*/` closes the comment early, leaving `still-outer */` as invalid code.
- A `//` comment only lasts until the end of that physical line — if it accidentally continues onto the next line without another `//`, that next line is treated as real code again.
- Comments are completely removed during the **Preprocessing** stage (Topic 2) — they have zero effect on the compiled program's size or speed.
- Over-commenting obvious code (e.g. `i++; // increase i by one`) adds noise instead of value — good comments explain **why**, not just restate **what** the code already makes clear.

Why does "Documentation" matter beyond just comments?

Good documentation also includes meaningful variable names, consistent indentation, and a clear structure (Topic 1) — comments are one tool among several that make code understandable to someone else (or to you, six months later, having forgotten your own logic).

📖 **One-Line Revision:** `//` comments one line; `/* */` comments a block; both are ignored by the compiler and exist purely to make code understandable for humans.

C PROGRAMMING

05 - Tokens (Keywords, Identifiers, Constants, Strings)

🎯 **Aim:** To understand what a "token" is — the smallest meaningful piece of a C program — and to learn its four core categories.

1. WHAT IS A TOKEN?

When the compiler reads your source code, it doesn't see it as raw text — it breaks the line into the smallest individual units that still carry meaning, called **tokens**. Think of tokens the way words are the building blocks of a sentence: the compiler groups characters into tokens first, then figures out how those tokens combine into valid statements.

```
int age = 25;
```

This single line is broken by the compiler into **5 tokens**:

int → a **Keyword**

age → an **Identifier**

= → an **Operator**

25 → a **Constant**

→ a **Punctuator/Separator** (marks end of statement)

2. CATEGORY 1: KEYWORDS

Keywords are **reserved words** that already have a fixed, special meaning built into the C language. You cannot use a keyword as a variable name, function name, or any other identifier.

The 32 Standard C Keywords

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

🔔 Keywords are always written in **lowercase** in standard C, and you have already used many of them: int, float, char, double, return, if, while, for.

3. CATEGORY 2: IDENTIFIERS

Identifiers are the **names you invent** for variables, functions, arrays, and so on — like age, total, calculateArea.

✓ Valid Identifier Rules

- Can contain letters, digits, and underscore _
- Must **start** with a letter or underscore, never a digit
- Cannot be a reserved keyword
- Case-sensitive: Age and age are two different identifiers

- No spaces or special symbols like @, -, %

✗ Invalid Identifier Examples

- 2age → starts with a digit
- my-age → hyphen not allowed
- float → it's a keyword
- student age → contains a space
- total\$ → \$ is not permitted

4. CATEGORY 3: CONSTANTS

Constants are **fixed literal values** that do not change — they appear directly in the code as-is (covered in full depth in Topic 10).

Constant Type	Example
Integer Constant	25, -10, 0
Floating-Point Constant	3.14, -0.5
Character Constant	'A', '9', '\$'
String Constant	"Hello", "Ramya"

5. CATEGORY 4: STRINGS

A **string literal** is a sequence of characters enclosed in **double quotes** " ", treated by the compiler as a single token — even though it may contain multiple characters. Note the distinction from a single character constant, which uses **single quotes** ' '.

```
char grade = 'A'; // character constant – single quotes
char name[] = "Ramya"; // string literal – double quotes
```

'A' is stored as a single byte (its ASCII numeric code).

"Ramya" is actually stored as an **array of characters**: R, a, m, y, a, \0 — C automatically appends a hidden \0 (null character) to mark where the string ends.

One-Line Revision: A token is the smallest meaningful unit of code — Keywords (reserved words), Identifiers (your own names), Constants (fixed literal values), and Strings (quoted text) are four of the main token categories, alongside Operators and Punctuators.

C PROGRAMMING

06 - Data Types (Part 1 of 3): Overview & Basic Types

✦ **Aim:** To understand what a data type is and to learn the five basic data types every C program is built from.

1. WHAT IS A DATA TYPE?

Every variable in C must be given a **data type** before it can be used. The data type tells the compiler two things: **(1)** how many bytes of memory to reserve for the variable, and **(2)** what kind of value it is allowed to hold (a whole number, a decimal number, a single letter, etc.). Without a data type, the compiler would not know how to interpret the bits stored in memory.

Data Type	Used for	Example Value
int	Whole numbers (no decimal point)	20, -5, 1000
float	Decimal numbers, ~6 digits of accuracy	5.5, 3.14
double	Decimal numbers needing more accuracy	25000.75
char	A single character	'A', '9', '\$'
char name[50]	A group of characters (a word/string)	"Ramya"

💡 Why do we have both float and double?

Both store decimal numbers, but **double** reserves twice the memory (8 bytes vs 4 bytes) and can store a decimal value far more precisely (about 15 digits vs about 6 digits). Use **double** whenever accuracy matters, such as for money (salary) or scientific calculations.

2. THE TWO BROAD FAMILIES OF DATA TYPES

🔢 Integer Family

Store whole numbers only — no decimal point, ever.

int, short int, long int, long long int, and their unsigned versions. Every one of these has multiple "flavours" for controlling size and sign — covered in full in Part 2 of this topic.

🔢 Floating-Point Family

Store numbers that may include a decimal point.

float, double, and long double — differing only in how much memory (and therefore how much precision) each reserves.

What about char?

char technically belongs to the integer family too — internally, a character is just stored as a small integer (its ASCII code, e.g. 'A' is stored as 65). C simply lets us treat that number as a printable character using the %c format specifier.

3. WHAT YOU'LL LEARN NEXT

This topic continues in two more parts, both zooming into details this overview doesn't cover:

- **Part 2 (Topic 7):** the full family of integer size modifiers — short, long, long long, signed/unsigned — and their exact format specifiers.

- **Part 3 (Topic 8):** the exact memory size and numeric range of every data type, plus the `sizeof()` operator to verify sizes on your own machine.

 **One-Line Revision:** `int` → whole number, `float` → decimal, `double` → precise decimal, `char` → single character (really a small integer), `string` (`char` array) → group of characters.

C PROGRAMMING

07 - Data Types (Part 2 of 3): Integer Types & Size Modifiers

🔗 **What will we learn?** How C offers several "flavours" of the integer type (short, normal, long, long long — each signed or unsigned) so we can pick the right size for the numbers we need to store, without wasting memory.

1. WHY DO WE NEED DIFFERENT INTEGER SIZES?

A plain `int` is usually enough, but sometimes we need to save memory (e.g. storing thousands of small numbers, so `short` is enough) or we need to store numbers far bigger than `int` can hold (e.g. a national population, so we need `long long`). The **size modifiers** — `short`, `long`, `long long`, and `signed/unsigned` — let us control exactly how much memory a whole number uses and whether it can be negative.

2. INTEGER TYPES & MEANING

1. short int (Small whole number)

```
short int a = 10;
```

Format Specifier: `%hd`

2. unsigned short int (Positive only, small)

```
unsigned short int b = 50;
```

Format Specifier: `%hu`

3. int / signed int (Normal integer, most common)

```
int c = -200;
```

Format Specifier: `%d`

4. unsigned int (Positive normal integer)

```
unsigned int d = 400;
```

Format Specifier: `%u`

5. long int (Large whole number)

```
long int e = 100000L;
```

Format Specifier: `%ld`

6. unsigned long int (Large positive only)

```
unsigned long int f = 200000UL;
```

Format Specifier: `%lu`

7. long long int (Very large whole number)

```
long long int g = 9000000000LL;
```

Format Specifier: `%lld`

8. unsigned long long int (Very large, positive only)

```
unsigned long long int h = 18000000000ULL;
```

Format Specifier: `%llu`

🔗 What do "signed" and "unsigned" actually mean?

Every integer type reserves a fixed number of bits. **signed** (the default) splits that range between negative and positive numbers, so it can represent both. **unsigned** gives up the ability to store negatives, and in exchange uses that freed-up range to store *larger positive* numbers — roughly double the positive range of the signed version. Use unsigned only when you are certain the value can never be negative (e.g. a count of students, an array index, a byte size).

3. SIZE & RANGE TABLE (TYPICAL MODERN 64-BIT SYSTEM, E.G. GCC ON LINUX)

Data Type	Size (Bytes)	Format	Range
-----------	--------------	--------	-------

short int	2	%hd	-32,768 to 32,767
unsigned short int	2	%hu	0 to 65,535
int	4	%d	-2,147,483,648 to 2,147,483,647
unsigned int	4	%u	0 to 4,294,967,295
long int	8 (4 on some older/Windows compilers)	%ld	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (on 8-byte systems)
unsigned long int	8 (4 on some older/Windows compilers)	%lu	0 to 18,446,744,073,709,551,615 (on 8-byte systems)
long long int	8	%lld	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long int	8	%llu	0 to 18,446,744,073,709,551,615

⚠ **Correction / Important Note:** The exact byte size of `int` and `long int` is technically **compiler and system dependent** — the C standard only guarantees a *minimum* size. On almost every modern system you will use (64-bit Linux/GCC, which Voflic students typically compile on), `int` is 4 bytes and `long int` is 8 bytes, as shown above. Always confirm with the `sizeof()` operator (covered in the next topic) rather than memorising numbers blindly.

4. SIMPLE C PROGRAM & OUTPUT

```
#include <stdio.h>

int main() {
    short int a = 10;
    int b = 100;
    long int c = 1000;
    long long int d = 10000;

    unsigned int e = 500;

    printf("short int = %hd\n", a);
    printf("int = %d\n", b);
    printf("long int = %ld\n", c);
    printf("long long int = %lld\n", d);
    printf("unsigned int = %u\n", e);

    return 0;
}
```

Output:

```
short int = 10
int = 100
long int = 1000
long long int = 10000
unsigned int = 500
```

Note:

Even though these small values (10, 100, 1000...) would fit into any of the types, declaring them as `short`

, `long`

, `long long`

here is only to demonstrate the correct

declaration syntax

and matching
format specifier

for each type — not because these particular numbers need that much space.

Why does the literal number need a suffix like L, UL, LL, ULL?

By default, a plain number like 100000 is treated by the compiler as an int. If that number is too big to fit in an int but we are assigning it to a long long, the suffix tells the compiler "treat this literal itself as a long/long long/unsigned value" so no data is lost during the calculation, even before it reaches the variable.

- 100000L → treat as long
- 200000UL → treat as unsigned long
- 9000000000LL → treat as long long
- 18000000000ULL → treat as unsigned long long

🔔 Important Rules to Remember:

- **signed** = stores both negative and positive numbers (this is the default if you don't write "unsigned").
- **unsigned** = stores ONLY positive numbers (0 and above), which roughly doubles the maximum positive value compared to the signed version.
- Always match the printf()/scanf() format specifier to the exact variable type — mismatching them (e.g. printing a long long with %d) gives wrong/garbage output, especially for large numbers.
- Use the correct suffix for large literals: L (long), UL (unsigned long), LL (long long), ULL (unsigned long long).

C PROGRAMMING

08 - Data Types (Part 3 of 3): Size, Range & sizeof()

📌 **What are Data Types (recap)?** A data type tells the compiler what kind of value a variable stores and exactly how many bytes of RAM to reserve for it. This page is the master reference table for every basic type in C, plus the `sizeof()` operator that lets us verify these sizes on our own machine.

1. COMPLETE QUICK-REVISION TABLE

Data Type	Memory Size	Format	Range / Purpose
char	1 byte	%c	Compiler-dependent: usually -128 to 127 (signed) — stores 1 character
signed char	1 byte	%c	-128 to 127
unsigned char	1 byte	%c	0 to 255 (positive only)
short int	2 bytes	%hd	-32,768 to 32,767 (small integers)
unsigned short int	2 bytes	%hu	0 to 65,535
int / signed int	4 bytes	%d	-2,147,483,648 to 2,147,483,647
unsigned int	4 bytes	%u	0 to 4,294,967,295
long int	4 or 8 bytes (system-dependent)	%ld	System-dependent — up to $\pm 9.22 \times 10^{18}$ on 8-byte systems
unsigned long int	4 or 8 bytes (system-dependent)	%lu	System-dependent — up to 18.4×10^{18} on 8-byte systems
long long int	8 bytes	%lld	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long int	8 bytes	%llu	0 to 18,446,744,073,709,551,615
float	4 bytes	%f	~6 decimal digits precision
double	8 bytes	%lf	~15 decimal digits precision
long double	16 bytes (commonly, though 10/12 bytes on some compilers)	%Lf	Higher precision than double

⚠️ **Correction / Clarification:** Plain `char`'s signedness (whether it behaves like `signed char` or `unsigned char`) is left to the compiler by the C standard — on most common compilers (including GCC on x86/Linux) it behaves as **signed**, so we list its range as -128 to 127. Never rely on this if a value must reliably be treated as positive-only — declare `unsigned char` explicitly instead.

2. FINDING SIZE USING THE sizeof() OPERATOR

```
#include <stdio.h>

int main() {
    printf("char = %zu bytes\n", sizeof(char));
    printf("short int = %zu bytes\n", sizeof(short int));
    printf("int = %zu bytes\n", sizeof(int));
    printf("long int = %zu bytes\n", sizeof(long int));
    printf("long long int = %zu bytes\n", sizeof(long long int));
    printf("float = %zu bytes\n", sizeof(float));
    printf("double = %zu bytes\n", sizeof(double));
    printf("long double = %zu bytes\n", sizeof(long double));

    return 0;
}
```

sizeof() is a built-in **compile-time** operator in C — the compiler computes the answer while compiling, not while the program runs, so it costs nothing at run-time.

It returns the exact memory size of a variable or data type, in bytes, for *your specific compiler and system* — this is the most reliable way to check sizes rather than assuming.

Its return type is `size_t`, an unsigned integer type — always print it with `%zu` (not `%d`) to avoid warnings/undefined behaviour.

`sizeof` can be used on a type name, e.g. `sizeof(int)`, or directly on a variable, e.g. `sizeof(age)` — both work.

Expected Sample Output (on a typical 64-bit system):

```
char = 1 bytes
short int = 2 bytes
int = 4 bytes
long int = 8 bytes
long long int = 8 bytes
float = 4 bytes
double = 8 bytes
long double = 16 bytes
```

★ **The 4-Point Rule for Every Type:**
Data Type → **Memory Size** → **Range** → **Format Specifier**

💡 Why should we care about size and range at all?

- Picking a type too small for the data causes **overflow** — the value silently wraps around to an incorrect number instead of giving an error.
- Picking a type too large wastes memory — usually harmless for a small student program, but matters a lot in large-scale software or embedded systems with limited RAM.
- Understanding size is the foundation for later topics like pointers, arrays, and structures, where the exact byte layout of memory matters.

C PROGRAMMING

09 - Variables

✦ **Aim:** To understand what a variable is, how to declare and initialise one, the rules for naming them, and the idea of variable scope.

1. WHAT IS A VARIABLE?

A variable is a **named location in memory** that can store a value which may change while the program runs — hence "vari-able." Every variable in C needs a **data type** (Topic 6) so the compiler knows how many bytes to reserve and how to interpret them, and a **name (identifier)** so your code can refer to it.

2. DECLARATION VS INITIALISATION

1. Declaration

Creates the variable and reserves memory, but doesn't give it a value yet.

```
int age;
```

age now exists in memory, but holds an unpredictable **garbage value** until something is stored in it.

2. Initialisation

Declares the variable AND gives it a starting value, in one step.

```
int age = 20;
```

This is the safer, more common style — the variable never has an unknown value.

```
int age; // declaration only
```

```
age = 20; // assignment – gives it a value later
```

```
int marks = 90; // declaration + initialisation together
```

```
int a = 5, b = 10, c = 15; // multiple variables, one line
```

Declaration and **assignment** can happen separately, as shown with age — first declared, then assigned a value on the next line.

Initialisation combines both into a single statement, as shown with marks — this is generally preferred since it avoids ever reading a garbage value by mistake.

C allows declaring **several variables of the same type** on one line, separated by commas, as shown with a, b, c.

3. RULES FOR NAMING A VARIABLE

A variable name is a kind of **identifier** (Topic 5) and must follow the same rules:

✓ Valid Variable Names

- age, total_marks, _count, studentAge
- Letters, digits, and underscore only

- Must start with a letter or underscore
- Case-sensitive: Age $\neq$ age

✗ Invalid Variable Names

- 2age → starts with a digit
- total marks → contains a space
- float → it's a reserved keyword
- total-marks → hyphen isn't allowed

💡 Good Naming Practice (beyond what the compiler enforces)

The compiler only cares about the rules above, but **readable** code also uses meaningful names. `int a;` compiles fine, but `int studentAge;` tells the next reader (including you, later) exactly what it's for — this is part of good "documentation" practice from Topic 4.

4. A FULL EXAMPLE

```
#include <stdio.h>

int main() {
    int age = 20;
    float height = 5.6;
    char grade = 'A';

    printf("Age: %d, Height: %.1f, Grade: %c\n", age, height, grade);

    age = 21; // the VALUE stored in age changes – that's what makes it a variable
    printf("Age after birthday: %d\n", age);

    return 0;
}
```

Output:

Age: 20, Height: 5.6, Grade: A
Age after birthday: 21

Notice

age

holds

20

, then later holds

21

— the same named memory location, just updated with a new value.

5. A PREVIEW OF SCOPE (LOCAL VS GLOBAL)

Where a variable is declared decides where it can be used — this is called its **scope**. This was briefly introduced in Topic 1 (Structure of a C Program) and will come up often going forward:

Type	Declared	Accessible From
Local Variable	Inside a function (e.g. inside <code>main()</code>)	Only within that function
Global Variable	Outside every function, at the top of the file	Any function in the file

📝 **One-Line Revision:** A variable = a named, typed memory location whose value can change; declare it with a type and a valid identifier name, and prefer initialising it immediately to avoid garbage values.

C PROGRAMMING

10 - Constants

🎯 **Aim:** To understand what a constant is, the different ways C lets us define one, and when to use each method.

1. WHAT IS A CONSTANT?

A constant is a value that **cannot be changed** once the program is compiled — the opposite of a variable, whose value can change while the program runs. Constants are useful for values that are fixed by definition (like 3.14159 for π) or values you want to protect from being accidentally modified later in the code.

2. THREE WAYS TO DEFINE A CONSTANT IN C

1. Literal Constant

A fixed value written directly into an expression — the simplest kind, already used throughout every earlier topic.

```
int total = a + 10;
```

Here 10 is a literal integer constant.

2. const Keyword

Declares a normal, typed variable that the compiler will refuse to let you modify after initialisation.

```
const float PI = 3.14159;
```

Any later attempt like `PI = 3.2;` causes a **compile-time error**.

3. #define Preprocessor Macro

Defines a symbolic name that the **preprocessor** (Topic 2) textually replaces with the given value, everywhere it appears, before compilation even starts.

```
#define PI 3.14159

int main() {
    float area = PI * r * r;
    ...
}
```

Before compilation, every occurrence of `PI` is literally swapped for 3.14159 in the source text — `PI` itself has no type and is not really a "variable" at all.

3. CONST VS #DEFINE — WHICH ONE TO USE?

Feature	const	#define
Processed by	Compiler	Preprocessor (before compilation)
Has a data type	Yes (e.g. float, int)	No — pure text substitution
Follows scope rules	Yes (local/global like a variable)	No — applies everywhere below it in the file

Uses memory	Yes, like a normal variable	No, it doesn't exist after preprocessing
Type-checked by compiler	Yes — safer	No — can cause subtle bugs
Recommended for	Most modern C code (safer, debuggable)	Simple global constants, especially in older-style code

🔗 **Modern Recommendation:** Prefer `const` over `#define` when possible. Because `const` variables have a real type and obey scope, the compiler can catch type-mismatch mistakes that `#define` would silently let through (since `#define` is just blind text replacement, with zero type awareness).

4. CONSTANT TYPES (RECAP FROM TOKENS)

Type	Example	Notes
Integer Constant	10, -25, 0	No decimal point
Real / Floating-Point Constant	3.14, -0.5	Has a decimal point
Character Constant	'A', '7'	Single character, single quotes
String Constant	"Voflic"	Multiple characters, double quotes

Common beginner slip: writing `#define PI 3.14159;` with a trailing semicolon. Since `#define` is pure text replacement, that semicolon gets copied in too — turning `area = PI * r * r;` into `area = 3.14159; * r * r;`, which is a syntax error. `#define` lines should **not** end with a semicolon.

📖 **One-Line Revision:** A constant's value never changes — write it directly as a literal, protect a typed variable with `const`, or define a text macro with `#define` (no semicolon, no type-checking).

C PROGRAMMING

TOPIC 11: TYPE CASTING (Implicit & Explicit)

🎯 **Aim:** To understand what type casting is, the difference between implicit and explicit casting, and how to avoid the classic "integer division" trap that silently produces wrong results.

1. WHAT IS TYPE CASTING?

Type casting is the process of converting a value from one data type into another — for example, converting an `int` into a `float`, or a `float` into an `int`. This is needed because C is strict about types: mixing types in a calculation or assignment requires the value to first be converted so both sides "speak the same language." There are two ways this conversion can happen:

- **Implicit type casting** — the compiler converts it automatically, without you asking.
- **Explicit type casting** — you, the programmer, manually force the conversion.

⚙️ 1. Implicit Type Casting (Automatic)

Performed automatically by the C compiler, usually when a "smaller" type is combined with or assigned to a "larger" type — this is called **type promotion**. No data is lost in this direction.

```
int a = 10;
float b = a; // Auto converted to float
```

`int (10) → Compiler → float (10.000000)`

```
Output: Integer value: 10
Float value: 10.000000
```

✍️ 2. Explicit Type Casting (Manual)

The programmer forces conversion using the cast operator `(type)value`. Used when going from a "larger" type to a "smaller" one, where data (like the decimal part) may be lost — the compiler will not do this automatically because it could silently lose information, so it requires an explicit request.

```
float a = 10.75;
int b = (int)a; // Drops .75 decimal
```

`float (10.75) → (int) → int (10)`

```
Output: Float value: 10.750000
Integer value: 10
```

Why does casting float to int give 10, not 11?

Explicit `(int)` casting always **truncates** — it simply chops off everything after the decimal point. It does **not** round to the nearest whole number. So `(int)10.75` gives 10, and even `(int)10.99` would still give 10, not 11.

Casting Type	Who Performs It?	Syntax / Example	Key Takeaway
Implicit	Compiler	<code>float b = a;</code>	Automatic promotion, safe (no data loss going small → large)
Explicit	Programmer	<code>int b = (int)a;</code>	Manual conversion operator, may lose data (large → small)

2. THE CLASSIC TRAP: TYPE CASTING IN DIVISION

Why does `5 / 2` output `2.000000` instead of `2.500000` in C, even when the result is stored in a `float`?

✗ Without Casting (Integer Division)

```
int a = 5, b = 2;
float c;
c = a / b;
```

Evaluation steps:

1. `a / b` is calculated **first**, before any assignment happens.
2. Since both `a` and `b` are `int`, C performs **integer division**: `5 / 2` mathematically is `2.5`, but integer division discards the decimal part, giving `2`.
3. Only **after** this truncated integer result (`2`) is computed does the assignment to the float variable `c` happen — so `c` becomes **`2.000000`**, and the `.5` is already permanently lost by that point.

Output: `2.000000`

✓ With Explicit Casting (Floating Division)

```
int a = 5, b = 2;
float c;
c = (float)a / b;
```

Evaluation steps:

1. `(float)a` converts `a` from `5` (`int`) to `5.0` (`float`) **before** the division happens.
2. Now the expression is `5.0 / 2` — since one operand is already a `float`, C automatically promotes `b` to `float` as well and performs **floating-point division**.
3. Result is the mathematically correct **`2.500000`**, stored fully in `c`.

Output: `2.500000`

💡 **Core Teaching Rule:** Casting just **one operand** before an arithmetic calculation (e.g., `(float)a / b`) is enough to make the **entire expression** evaluate as floating-point — C automatically promotes the other operand to match. You do NOT need to cast both sides.

⚠ Common Beginner Mistakes with Casting

- Writing `c = (float)(a / b);` instead of `c = (float)a / b;` — this casts the already-truncated integer result to float too late; you still get `2.000000`, not `2.500000`. The cast must happen **before** the division, on one of the operands.
- Assuming `(int)` rounds — it truncates (always rounds toward zero), which is not the same as rounding to the nearest whole number.
- Forgetting that casting a double down to float, or a large long long down to int, can lose precision or overflow — explicit casting removes the compiler's safety warning, so the responsibility shifts to the programmer.

3. QUICK MEMORY FLOWCHART

IMPLICIT → Automatic → Compiler decides

EXPLICIT → Manual → Programmer writes (type)

📖 **One-Line Revision:** Implicit = automatic, safe, compiler-driven conversion | Explicit = manual, programmer-driven conversion using `(type)value` | `(float)5 / 2 = 2.5`, but `5 / 2 = 2`.

C PROGRAMMING

12 - Operators & Expressions in C

🔗 **What are Operators?** Special symbols that perform an operation on one or more values (called operands). For example in $a + b$, $+$ is the operator and a, b are the operands.

1. OPERATORS SUMMARY

Category	Operators	Example
Arithmetic	$+$ $-$ $*$ $/$ $\%$	$10 \% 3 = 1$
Relational	$>$ $<$ $>=$ $<=$ $==$ $!=$	$5 > 2 \rightarrow 1$ (True)
Logical	$\&\&$ $\ \ $ $!$	$(5 > 2) \&\& (3 > 1) \rightarrow 1$ (True)
Assignment	$=$ $+=$ $-=$ $*=$ $/=$	$a += 5$ means $a = a + 5$
Increment / Decrement	$++$ $--$	$count++$ / $++count$
Conditional (Ternary)	$condition ? val1 : val2$	$(a > b) ? a : b$
Bitwise	$\&$ $ $ $^$ $<<$ $>>$	$a \& b$

2. THE FULL PROGRAM

```
#include <stdio.h>

int main() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

    // ... Arithmetic, Relational, Logical, Assignment,
    // Increment/Decrement, Conditional, Bitwise sections below

    return 0;
}
```

int a, b; → Declares two integer variables in a single line (comma-separated).

scanf("%d%d", &a, &b); → Reads two integers in one call. The user can separate them by a space, tab, or Enter — `scanf()` automatically skips whitespace between numeric inputs.

The rest of the program (shown section by section below) demonstrates every operator category using the same a and b .

3. ARITHMETIC OPERATORS

Perform standard mathematical calculations.

```
printf("Addition (+): %d\n", a + b);
```

```
printf("Subtraction (-): %d\n", a - b);
printf("Multiplication (*): %d\n", a * b);
printf("Division (/): %d\n", a / b);
printf("Modulus (%): %d\n", a % b);
```

/ (**Division**) → When both operands are int, C performs **integer division** — the decimal part is dropped, not rounded.

% (**Modulus**) → Gives the **remainder** after division. Works only with integer operands, never with float/double.

Note the double %% in the printf string — since % normally starts a format specifier, we must write it twice to print a literal % symbol.

4. RELATIONAL OPERATORS

Compare two values and produce a **Boolean-style result**: C has no separate boolean type, so it uses 1 for true and 0 for false.

```
printf("%d > %d = %d\n", a, b, a > b);
printf("%d < %d = %d\n", a, b, a < b);
printf("%d >= %d = %d\n", a, b, a >= b);
printf("%d <= %d = %d\n", a, b, a <= b);
printf("%d == %d = %d\n", a, b, a == b);
printf("%d != %d = %d\n", a, b, a != b);
```

== vs = → A very common beginner bug: == compares two values for equality; a single = assigns a value. Using = inside an if condition by mistake silently changes the variable instead of comparing it.

Each result printed above is either 1 (condition is true) or 0 (condition is false).

5. LOGICAL OPERATORS

Combine multiple conditions into one. Also give results as 1 or 0.

```
printf("(a > 0 && b > 0) = %d\n", a > 0 && b > 0);
printf("(a > 0 || b > 0) = %d\n", a > 0 || b > 0);
printf("(!(a > b)) = %d\n", !(a > b));
```

&& (Logical AND) → True only if **both** sides are true.

|| (Logical OR) → True if **at least one** side is true.

! (Logical NOT) → Reverses/flips the result: true becomes false and false becomes true.

6. ASSIGNMENT OPERATORS

Shorthand ways to update a variable's value based on itself.

```
int c = a;

c += b; // same as c = c + b
printf("c += b: %d\n", c);

c = a; c -= b; // c = c - b
c = a; c *= b; // c = c * b
c = a; c /= b; // c = c / b
```

Each compound operator (+=, -=, *=, /=, and also %=) is exactly equivalent to the longer "variable = variable OP value" form, just shorter to type and slightly clearer to read once you're used to it.

Notice `c` is reset to `a` before each new example — otherwise each operation would build on the previous result.

7. INCREMENT AND DECREMENT OPERATORS

Quickly add or subtract 1 from a variable. Available in two forms: **prefix** (`++c`) and **postfix** (`c++`).

```
c = a;
printf("c++ = %d\n", c++);
printf("After c++ = %d\n", c);

printf("++c = %d\n", ++c);

printf("c-- = %d\n", c--);
printf("After c-- = %d\n", c);

printf("--c = %d\n", --c);
```

Postfix `c++` → Uses the **current** value of `c` in the expression first, THEN increases it by 1 afterwards.

Prefix `++c` → Increases `c` by 1 FIRST, then uses the new value in the expression.

This ordering difference only matters when the increment is used *inside* a larger expression (like a `printf` argument); on its own line, `c++`; and `++c`; behave identically.

The same prefix/postfix logic applies to `--` (decrement).

8. CONDITIONAL (TERNARY) OPERATOR

A compact one-line replacement for a simple if-else, using the pattern `condition ? value_if_true : value_if_false`.

```
int max = (a > b) ? a : b;
printf("Maximum = %d\n", max);
```

It is called "ternary" because it is the only C operator that takes **three** operands: the condition, the true-value, and the false-value.

Equivalent longer form:

```
if (a > b) { max = a; } else { max = b; }
```

9. BITWISE OPERATORS

Operate directly on the individual **binary bits** of integer values. Used heavily in embedded programming, flags/permissions, and performance-critical code.

```
printf("a & b = %d\n", a & b);
printf("a | b = %d\n", a | b);
printf("a ^ b = %d\n", a ^ b);
printf("a << 1 = %d\n", a << 1);
printf("a >> 1 = %d\n", a >> 1);
```

& (Bitwise AND) → Compares each bit; result bit is 1 only if both input bits are 1.

| (Bitwise OR) → Result bit is 1 if at least one input bit is 1.

^ (Bitwise XOR) → Result bit is 1 only if the two input bits are different.

<< (Left Shift) → Shifts all bits left by the given amount — each left-shift by 1 effectively doubles the value.

>> (Right Shift) → Shifts all bits right — each right-shift by 1 effectively halves the value (integer division by 2).

Don't confuse `&` (bitwise AND, works on numbers) with `&&` (logical AND, works on true/false conditions) — mixing them up is a very common bug.

them up is a very common bug.

⚠ **Important Practical Notes:**

- For division (a / b) and modulus ($a \% b$), b must never be **0** — dividing by zero causes undefined behaviour / a crash.
- Modulus $\%$ works **ONLY** with integer operands, never float or double.
- Always double-check $==$ vs $=$ and $\&$ vs $\&\&$ — these are the most common operator mistakes for beginners.

C PROGRAMMING

TOPIC 13: OPERATOR PRECEDENCE

🎯 **Aim:** To understand how operator precedence and associativity determine the exact evaluation order of an expression that mixes multiple operators, and how parentheses let us override the default order.

1. WHAT IS OPERATOR PRECEDENCE?

When an expression contains more than one operator, C does not simply evaluate left to right — it follows a strict priority order called **operator precedence**, which decides **which operator is evaluated first**. Operators with higher precedence "bind tighter" and are calculated before operators with lower precedence, exactly like the "BODMAS"/"PEMDAS" rule you learnt in mathematics (multiplication before addition, for example).

```
#include <stdio.h>

int main() {
    int result;
    result = 10 + 5 * 2;
    printf("%d", result);
    return 0;
}
```

Output & Trace:

Step 1:

*

has higher precedence than

+

, so

5 * 2

is calculated first →

10

Step 2: Then the addition happens →

10 + 10

→

20

Final Output:

20

What if we actually wanted addition to happen first?

Wrap the part you want evaluated first inside parentheses: `result = (10 + 5) * 2;` → now 10 + 5 is forced to happen first (**15**), then multiplied by 2, giving **30** instead of 20. Parentheses always have the **highest** precedence of all, which is exactly why they are the standard tool for making your intended evaluation order explicit and bug-free.

2. WHAT IS ASSOCIATIVITY?

Precedence decides *which operator* goes first when operators **differ**. But what happens when two operators of the **same** precedence appear next to each other, like `20 - 5 - 3`? That's where **associativity** comes in — it decides the **direction** (left-to-right or right-to-left) in which same-priority operators are grouped. `20 - 5 - 3` is left-to-right associative, so it's grouped as `(20 - 5) - 3 = 12`, not `20 - (5 - 3) = 18`.

3. PRECEDENCE & ASSOCIATIVITY HIERARCHY TABLE

Priority	Operator Category	Operators	Associativity
1 (Highest)	Parentheses / Postfix	() [] . -> ++(postfix) --(postfix)	Left to Right
2	Unary / Prefix	+ - ! ~ ++(prefix) --(prefix) sizeof & *	Right to Left
3	Multiplicative	* / %	Left to Right
4	Additive	+ -	Left to Right
5	Shift	<< >>	Left to Right
6	Relational	< <= > >=	Left to Right
7	Equality	== !=	Left to Right
8	Bitwise AND	&	Left to Right
9	Bitwise XOR	^	Left to Right
10	Bitwise OR		Left to Right
11	Logical AND	&&	Left to Right
12	Logical OR		Left to Right
13	Ternary (Conditional)	?:	Right to Left
14 (Lowest)	Assignment	= += -= *= /= %=	Right to Left

⚠ **Correction / Extra Detail:** The original quick-reference table only listed 10 broad groups and merged all bitwise operators together with logical operators. In real C precedence rules, bitwise AND (&), XOR (^), and OR (|) each sit at their **own, separate** priority level — all below relational/equality but above logical &&||. The ternary operator ?: is also lower priority than all of those, and assignment is the lowest of all (besides the comma operator, which is beyond this topic). The full 14-row table above is the complete and accurate version.

4. WORKED EXAMPLE USING BOTH RULES TOGETHER

Consider: `int x = 2 + 3 * 4 > 10 && 5 == 5;`

Step 1 (Multiplicative, priority 3): `3 * 4 → 12`

Step 2 (Additive, priority 4): `2 + 12 → 14`

Step 3 (Relational, priority 6): `14 > 10 → 1 (true)`

Step 4 (Equality, priority 7): `5 == 5 → 1 (true)`

Step 5 (Logical AND, priority 11): `1 && 1 → 1 (true)`

Step 6 (Assignment, lowest priority): `x = 1`

💡 Pro Tip — Don't Memorise, Use Parentheses:

Even experienced programmers rarely memorise every row of this table. The safest and most readable habit is: whenever an expression mixes more than two different operator types, add parentheses to make your intended order **explicit** — e.g. `(10 + 5) * 2` instead of relying on everyone remembering that * beats +. This prevents bugs and makes code easier for others (and your future self) to read.

Associativity = which direction to group when they're equal | Parentheses always win and should be used generously for clarity.

C PROGRAMMING

14 - Input/Output Functions

✦ **Aim:** To understand what Input/Output (I/O) functions are in C, where they come from, and the main functions available — before diving into the detailed formatted versions in the next topic.

1. WHAT IS AN INPUT/OUTPUT FUNCTION?

A program is only useful if it can exchange data with the outside world: reading what the user types (**input**) and showing results back (**output**). C does not build this ability into the language itself — instead, it is provided through **library functions** declared in the header file `stdio.h` ("Standard Input Output"), which is why every program that reads or displays anything begins with `#include <stdio.h>` (Topic 1).

2. THE MAIN I/O FUNCTIONS IN C

Function	Direction	Reads/Writes	Notes
<code>printf()</code>	Output	Formatted text of any type	The most-used output function; covered in depth next topic
<code>scanf()</code>	Input	Formatted values of any type	The most-used input function; covered in depth next topic
<code>getchar()</code>	Input	A single character	Simplest possible input — no format specifier needed
<code>putchar()</code>	Output	A single character	Simplest possible output — no format specifier needed
<code>gets()</code> (unsafe, avoid)	Input	A full line of text, including spaces	Removed from modern C standards due to buffer-overflow risk
<code>fgets()</code>	Input	A full line of text, safely, up to a size limit	The modern, safe replacement for <code>gets()</code>
<code>puts()</code>	Output	A string, followed automatically by a newline	Simpler than <code>printf()</code> when no formatting is needed

3. SIMPLE I/O WITHOUT ANY FORMAT SPECIFIERS

```
#include <stdio.h>

int main() {
    char ch;

    puts("Type a character: ");
    ch = getchar(); // reads exactly one character

    puts("You typed:");
    putchar(ch); // writes exactly one character
```



```
return 0;  
}
```

getchar() and **putchar()** work with a **single character** at a time — no format string like "%c" needed, since there's only one possible type they can handle.

puts() prints a string and automatically adds a newline afterward — unlike **printf()**, which requires you to add `\n` yourself.

These simpler functions are useful when you don't need C's flexible formatting — just plain text or a single character.

⚠ **Why avoid gets()?**

gets() reads a whole line but has **no way to limit how many characters** it accepts — if the input is longer than the destination array, it silently overflows into other memory, a serious security risk. Modern C standards have removed it entirely. Use **fgets()** instead, which takes a maximum size argument.

Why is printf()/scanf() so much more common than the simple functions?

getchar()/putchar() only handle single characters, and **puts()** only handles plain strings. Real programs constantly need to read/display **numbers** (int, float, double) alongside text — that flexibility is exactly what **printf()** and **scanf()** provide through **format specifiers**, which is the focus of the next topic.

📖 **One-Line Revision: I/O functions live in `stdio.h` and move data between your program and the user — `getchar()/putchar()/puts()` handle simple cases, while `printf()/scanf()` handle everything via formatted specifiers (next topic).**

C PROGRAMMING

15 - Formatted Input and Output (printf, scanf)

✦ **Aim:** To master `scanf()` and `printf()` — the two workhorse I/O functions from Topic 14 — using format specifiers to read and display every data type correctly.

1. WHAT DOES "FORMATTED" MEAN?

`printf()` and `scanf()` are called **formatted** I/O functions because they use a **format string** containing special codes — **format specifiers** — that tell C exactly what data type to expect at each point: **%d** for int, **%f** for float, **%lf** for double, **%c** for char, and **%s** for a string. This is what lets a single function handle every data type from Topics 6–8, unlike `getchar()/putchar()` from Topic 14.

2. A FULL EXAMPLE PROGRAM

```
#include <stdio.h>

int main() {
    int age;
    float height;
    double salary;
    char grade;
    char name[50];

    printf("Enter your name: ");
    scanf("%s", name);
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Enter your height: ");
    scanf("%f", &height);
    printf("Enter your salary: ");
    scanf("%lf", &salary);
    printf("Enter your grade: ");
    scanf(" %c", &grade);

    printf("\n----- Student Details ----- \n");
    printf("Name: %s\n", name);
    printf("Age: %d\n", age);
    printf("Height: %.2f\n", height);
    printf("Salary: %.2lf\n", salary);
    printf("Grade: %c\n", grade);

    return 0;
}
```

All five variables were declared following the rules from Topic 9 (Variables), each with the data type it needs from Topics 6–8.

Each `scanf()/printf()` pair uses the format specifier that matches its variable's exact type — mismatching them is one of the most common beginner bugs.

3. GETTING INPUT WITH SCANF()

We use **scanf()**; together with a **format specifier** that matches the variable's data type: **%d** for int, **%f** for float, **%lf** for double, **%c** for char, and **%s** for a string (character array).

★ **Why do we use & before the variable name?**

scanf() does not simply store a value into a variable — it needs to know the exact **memory address** where the value should be written. The & ("address-of") operator gives it that address. A string/array is a special case: the array name (name) already represents the address of its first element, so we do NOT add & before it.

Easy Rule:

- scanf() + normal variable (int, float, double, char) → **use &** → e.g. &age
- scanf() + string/character array → **don't use &** → e.g. name

Why is there a space before %c in scanf(" %c", &grade);?

Every time the user types a value and presses **Enter**, the Enter key itself leaves a leftover newline character (\n) sitting in the input buffer. If we write plain "%c", scanf() would instantly read that leftover \n instead of waiting for the user's actual character. The single leading space tells scanf() to first skip over any whitespace (spaces, tabs, newlines) before reading the real character. This trick is usually only needed for %c; %d, %f, and %lf already skip leading whitespace automatically.

4. DISPLAYING OUTPUT WITH PRINTF()

We use **printf()**; to send output to the screen.

How does printf() work?

- printf("hi"); → Prints a **plain message** exactly as written.
- printf("Age: %d\n", age); → Prints the text "Age: ", then substitutes the value stored inside age in place of %d, then moves to a new line because of \n.

What does %.2f mean?

The number between % and the specifier is the **precision** — it fixes how many digits appear after the decimal point. %.2f rounds the value to exactly **2 decimal digits**.

Example: a stored value of 5.6789 is printed as 5.68 (rounded, not truncated).

⚠ **Common Beginner Mistakes with Formatted I/O**

- Using %d to print a float or double (or vice-versa) — this reads the memory bits incorrectly and prints garbage values. The specifier must always match the variable's type.
- Forgetting & before a normal variable in scanf() — the program may crash or silently fail to store the value, because scanf() was given a value instead of an address.
- Using char name[50] but typing a name longer than 49 characters — this overflows the array and can corrupt nearby memory. Always size arrays generously.
- %s with scanf() stops reading at the first space, so it cannot capture a full name with a space in it (e.g. "Ramya M"). That needs fgets() from Topic 14, not scanf("%s", ...).

5. QUICK REFERENCE TABLE

Variable	Declaration	scanf()	printf()
Integer	int age;	%d	%d
Float	float height;	%f	%f / %.2f
Double	double salary;	%lf	%lf / %.2lf
Character	char grade;	%c (note the space)	%c
String	char name[50];	%s (no &)	%s

📖 **One-Line Revision: Match the format specifier to the exact data type every time —**

%d/int, %f/float, %lf/double, %c/char, %s/string — and remember & for scanf() on everything except strings.

C PROGRAMMING

16 - Basic Error Messages and Debugging Techniques

🎯 **Aim:** To recognise the main categories of errors a C program can have, read a real compiler error message correctly, and build simple habits for finding and fixing bugs.

1. THE FOUR CATEGORIES OF ERRORS

1. Syntax Error

Breaks the grammar rules of C (e.g. missing semicolon). Caught at **compile-time** — the program will not even build.

2. Semantic / Type Error

Grammatically valid but logically inconsistent (e.g. mismatched format specifier). Often caught as a compiler **warning**.

3. Linker Error

The compiler understood everything, but a referenced function/variable was never actually defined anywhere. Caught at the **linking** stage (Topic 2).

4. Run-Time Error

The program compiles and starts, but crashes or misbehaves *while running* (e.g. divide by zero, array out of bounds).

5. Logical Error

The program runs completely fine, with no crash and no error message — but produces the **wrong answer** because of a mistake in the programmer's logic.

⚠️ **Logical errors are the hardest to catch** — the compiler cannot detect them at all, because nothing about the code is technically "wrong." Only careful testing (checking the output against what you expect by hand) reveals them.

2. READING A REAL COMPILER ERROR MESSAGE

```
int main() {  
    int age = 20  
    printf("%d", age);  
    return 0;  
}
```

GCC Output:

```
error.c:3:18: error: expected ';' before 'printf'  
  3 |     int age = 20  
    |             ^  
    |             ;
```

How to decode this message:

error.c:3:18 → the error is in file error.c, on **line 3**, around **column 18**. expected ';' before 'printf' → the compiler expected a semicolon right there, but instead found the start of the next statement. The ^ mark points to the exact spot. **Note:** the actual missing semicolon is at the *end of line 2* — compilers often report the error one token *after* where the real mistake is, so always check the line just above the reported one too.

3. COMMON BEGINNER ERRORS & FIXES

Symptom / Message	Likely Cause	Fix
expected ';' before ...	Missing semicolon at the end of the previous line	Add the missing ;
'age' undeclared	Variable used before being declared, or misspelled	Declare the variable first, check spelling/case
expected '}' at end of input	A curly brace { was never closed	Count braces carefully; use editor auto-indent to spot the gap
Warning: format %d expects int, got double	Format specifier doesn't match the variable's actual type	Match the specifier exactly: %d/%f/%lf/%c/%s
undefined reference to 'add'	Linker error — function was declared/called but never defined	Make sure the function body actually exists in the file
Program crashes / "Segmentation fault"	Run-time error — often array out-of-bounds or dividing by zero	Check array indices and guard against zero divisors
Output is wrong but no error shown	Logical error — flawed formula or condition	Trace through the logic by hand, or add debug prints (below)

4. SIMPLE DEBUGGING TECHNIQUES

🔍 1. Fix the First Error First

One real mistake often triggers a cascade of many reported errors.

Always scroll to the **very first** error in the compiler output and fix that one before worrying about the rest — recompiling often makes most of the later errors disappear on their own.

🔍 2. Compile Often, in Small Steps

Don't write 50 lines and compile once.

Compile after every small addition (a few lines at a time). This way, if an error appears, you know it must be in the few lines you just added — dramatically narrowing the search.

🔍 3. printf() Debugging

The simplest way to "see inside" a running program.

Temporarily add extra `printf("here, x = %d\n", x);` statements at key points to check whether variables hold the values you expect, at the moment you expect them to.

🔍 4. Read Errors Bottom-to-Top... Then Top-to-Bottom

Long error lists can be confusing.

A single typo can cause many follow-on errors below it. Read the **first** error message closely — it's usually the true root cause; later ones are often just side-effects of it.

💡 **Golden Debugging Habit:** Never guess-and-check randomly. Read the exact error message, find the exact file and line number it points to, understand *why* the compiler is confused there, and only then make a targeted fix — then recompile immediately to confirm.

📖 **One-Line Revision:** Syntax/Semantic errors are caught by the compiler, Linker errors at linking, Run-time errors while the program executes, and Logical errors give wrong output with no error at all — always fix the first reported error first, and use `printf()` to peek inside a running program when the bug isn't obvious.

